

CEquality

Scoring de crédit · Mobile Money Afrique de l'Ouest

Livrable 2 —Déploiement,Sécurité &Bilan

Master IA · 2iE · ECUE2 Sécurité & Coûts dans le Cloud

Juin 2026

01 CONTEXTE ET OBJECTIF

CEquality est une application de scoring de crédit ciblant les utilisateurs du service Wave Mobile Money en Afrique de l'Ouest. Ce Livrable 2 documente la mise en œuvre effective de l'architecture définie au Livrable 1 : conteneurisation des services, pipeline CI/CD avec scans de sécurité, et déploiement cloud.

La version actuelle est fonctionnelle en mode conteneurisé local et dispose d'une URL publique provisoire sur Render pendant que le provisionnement AWS est finalisé.

02 ARCHITECTURE FINALE

L'application est composée de trois services orchestrés par Docker Compose. Le frontend Angular est servi par Nginx, le backend FastAPI expose les endpoints ML et santé, et PostgreSQL assure la persistance.

Élément prévu L1	État	Commentaire
Frontend web conteneurisé	Oui	Angular + Nginx au lieu de React — écart documenté
Backend FastAPI	Oui	Service en place et testable
Base PostgreSQL	Oui	Volume Docker pour persistance locale
Déploiement cloud AWS	En cours	Structure prête ; provisionnement en finalisation
CI/CD avec scans sécurité	Oui	GitHub Actions : trufflehog, pip-audit, Trivy

03 DÉVELOPPEMENT

Backend FastAPI

- Route **/health** pour vérifier l'état du service.
- API **descore** exposée sous **/api/v1/predict/{user_id}**.
- Authentification JWT validée dans **app/core/security.py**.

- BaseSQLgéréeviaSQLAlchemy,initialisationdans **Backend/app/db.py**.

Frontend Angular 16

- Interface d'affichage du score et des résultats de prédiction.
- Consommation de l'API backend via un endpoint configurable.
- Build **npm run build** diffusé par Nginx dans Docker.

Machine Learning

- ModèleentraînésurdonnéesynthétiquesWave(quatre profils-types).
- Prédictionexécutéeenbackendvia**model.predict(...)**.
- Sortie : score de crédit, profil de risque, capacité d'emprunt.

04 CONTENEURISATION ET DEPLOIEMENT

Structure Docker

- **Backend/Dockerfile** — construit l'image FastAPI.
- **Frontend/Dockerfile** — build Angular + Nginx.
- **docker-compose.yml** — orchestre backend, frontend et db.

Ports locaux exposés :

frontend → <http://localhost:4200> · backend → <http://localhost:8000>

Pipeline CI/CD — GitHub Actions

- BuildetpushdesimagesversAmazon ECR.
- Déploiement cible : ECS Fargate.
- SecretsAWSinjectésviaGitHubSecrets (identité OIDC, aucune clé longue durée).

URL provisoire (Render)

Le déploiement AWS n'étant pas encore complètement finalisé, une URL publique est disponible sur Render pour la démonstration :

<https://cequality-frontend.onrender.com>

05 SECURITE — EXECUTION DU PLAN L1

Mesure prévue (L1)	État	Preuve / écart
Secrets en variables d'en	Oui	.env et secrets pipeline exclus du dépôt via .gitignore
Trufflehog en CI	Oui	Workflow ci-cd.yml exécute trufflehog à chaque push
pip-audit en CI	Oui	Audit Python activé sur Backend/requirements.txt
Trivy en CI	Oui	Scan image Docker inclus dans le pipeline
HTTPS en production	Partiel	HTTPS actif sur Render ; AWS en cours de finalisation
JWT auth API	Oui	Authentification implémentée dans app/core/security.py

Écart principal : le déploiement AWS n'est pas encore en production. L'URL Render garantit une démonstration HTTPS publique en attendant la finalisation du provisionnement.

06 COUTS — REEL VS ESTIME

Poste	Service	Estimé L1	Réel actuel
Compute backend ML	ECS Fargate 0.25 vCPU / 0.5 GB	~4 \$	0 \$ *
Frontend	Nginx ECS / S3 CloudFront	~1 \$	0 \$ *
Base de données	RDS db.t4g.micro (Free Tier)	0 – 15 \$	0 \$ *
Stockage images	Amazon ECR	~0,5 \$	0 \$ *
Bande passante	AWS Data Transfer < 1 GB	~0 \$	0 \$
Total		~5,5 \$	0 \$

** AWS non encore provisionné en production. Infrastructure locale Docker Compose sans coût. Démonstration publique sur plan Render gratuit. Une comparaison plan vs réalité sera produite après finalisation AWS.*

07 DIFFICULTES ET SOLUTIONS

Gestion des environnements

- Solution: Variables d'env centralisées dans .gitignore et GitHub Secrets.

Migration React → Angular

- Solution: Choix documenté comme écart L1 ; architecture conteneurisée identique.

Finalisation OIDC AWS

- Solution: Pipeline GitHub Actions structuré ; blocage sur configuration OIDC en cours de résolution.

08 CONCLUSION

CEquality est fonctionnel en mode conteneurisé local et dispose d'une démonstration publique accessible sur Render. Le pipeline CI/CD est opérationnel avec des scans de sécurité automatisés (trufflehog, pip-audit, Trivy) et une authentification JWT en place.

Le déploiement AWS (ECS Fargate, ECR, RDS) est structuré et prêt ; la finalisation du provisionnement permettra de livrer la production complète et d'établir un bilan de coûts réels.

URL provisoire active : <https://cequality-frontend.onrender.com>